CMPS4191 • LABORATORY 1

Measuring a Synchronous API Contract

An empirical investigation of request lifetime, work duration, and client waiting time

Duration	Work mode	Code
75 minutes	Pairs	Provided

Central question: How does report-generation time affect the externally observable behavior of a synchronous report API?

Learning outcomes

- Measure API response time under controlled conditions.
- Distinguish a prediction, an observation, and a causal explanation.
- Identify the relationship between work duration and response time.
- Explain how the synchronous contract couples request lifetime and work lifetime.
- Derive a system requirement from measured evidence.

The current contract

A contract is an externally observable promise between components. It states what a client may expect from a server; it does not describe the server's internal implementation.

Current promise: If the request succeeds, POST /v1/reports returns 200 OK with the completed report in the response body.

Because the completed report must be included in the response, the handler must finish generating the report before it can fulfill the contract. The client, therefore, waits while the work is performed.

Approximate relationship:

response time ≈ HTTP overhead + report-generation time

Experimental design

Research question

How does report-generation time affect the externally observable behavior of the synchronous report endpoint?

Hypothesis — complete before testing

If report-generation time increases, total response time will increase by approximately the same amount, because the server must complete report generation before it can write the HTTP response, leaving the client blocked for that entire duration.

Variables

Role	Variable
Independent	Artificial report-generation delay: 0 s, 3 s, 7 s, 12 s
Dependent	Response time, HTTP status, response bytes, complete report received, server completion log
Controlled	Endpoint, request body, consumer, reporting period, database contents, client command, and server timeout

Before you measure: make predictions.

Record predictions before running the server under each condition. Predictions are not graded for being correct; they are evidence of your reasoning before observation.

Delay	Predicted response time	Predicted status	Complete body?
0 s	0.01	generated	yes
3 s	3.01	generated	yes
7 s	7.1	generated	yes
12 s	12.5	failed	no

1. At what point do you predict the current design will fail or become unreliable? Why?

   We predicted it would fail at 12s because waiting that long for a response makes it seem like other requests are going in, which doesn't seem feasible. I think it will cancel the request because of the delay.

Procedure

Part A — Start one experimental condition

Your instructor will provide the database connection value. Start the API with one artificial delay condition:

```
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=0s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=3s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=7s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=12s
```

Controlled experiment: Change only the report-delay value. Restart the server between conditions. Keep the request, data, and measurement command unchanged.

Part B — Send and measure the request

In a second terminal, create request.json once:

```
{
  "consumer_id": "0198f000-0000-7000-8000-000000000001",
  "from": "2026-01-01T00:00:00Z",
  "to": "2027-01-01T00:00:00Z"
}
```

Then run the same measurement command for every condition:

```
curl --silent --show-error \
  --output response.json \
  --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n' \
  --request POST \
  --header 'Content-Type: application/json' \
  --data @request.json \
  http://localhost:4000/v1/reports
```

After every request, inspect the body:

```
cat response.json
```

Part C — Repeat consistently

1. Run the request three times for the current delay.
2. Record every result; do not discard unexpected results.
3. Inspect response.json and the server log after each request.
4. Stop and restart the server with the next delay.
5. Do not run different delay conditions simultaneously.

Raw measurements

Record the client-visible result and the server-visible result. A blank body, status 000, timeout, or connection error is still an observation and must be recorded.

Condition: 0 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	0.001388	369	completed	yes
2	200	0.001359	368	Completed	yes
3	200	0.001362	369	Completed	yes

Condition: 3 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	3.011587	369	Completed	yes
2	200	3.002791	369	Completed	yes
3	200	3.005392	369	Completed	yes

Condition: 7 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	7.011949	369	Completed	yes
2	200	7.008982	369	Completed	yes
3	200	7.009138	369	Completed	yes

Condition: 12 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	12.005353	363	Completed	yes
2	200	12.001865	364	Completed	yes
3	200	12.012636	364	Completed	yes

Summarize the evidence

For each condition, calculate the approximate mean response time:

$$\text{mean response time} = (T_1 + T_2 + T_3) \div 3$$

Delay	Mean response time	Successful responses	Complete reports
0 s	0.0014s	3 / 3	3/ 3
3 s	3.01s	3 / 3	3/ 3
7 s	7.01s	3 / 3	3/ 3
12 s	12.01s	3 / 3	3/ 3

Observation

Write only what you measured. Do not explain the mechanism yet.

When the report delay increased from 0s to 12s, the mean response time changed from approximately 0.0014s to 12.01s.

Explanation

Now explain the mechanism that produced the observations.

1. Why did the response time change as the artificial delay increased?

- The response time changed as the artificial delay increased because the handler called the function at the **time.sleep()**, this triggers the configured delay before the database is queried or writes any response. This is due to the sleep function being called synchronously within the request that is handling the goroutine; the client's connection stays open and waits for the entire duration; the response can't be sent until the sleep function and queries finish processing.

2. What must the handler complete before it can send 200 OK with the report?

- It must finish the artificial delay, execute the database query, and construct the JSON response body before it writes the 200 status and body to the client.

3. While the report is being generated, is the original request complete or outstanding?

- While the report is being generated, the original request remains outstanding because the HTTP connection remains open until all requests are finished.

4. Would asynchronous execution make the report run faster, or would it change when the client receives the acknowledgment?

- We believe that asynchronous execution would not execute the report faster, as the same amount of work will occur in the background. However, the client-side output will definitely change, as the client would be told that the request has been received and acknowledged immediately, rather than waiting for the report to finish generating. The actual report would continue to be processed in the background and provided once the work is complete.

Do not confuse cause with threshold. A 10-second write timeout may expose the problem, but 10 seconds is not a universal boundary. Increasing the timeout moves the boundary; it does not remove the coupling.

From evidence to requirements

A requirement describes a property the system must provide. Do not write implementation mechanisms such as 202, worker, queue, or polling in this section.

A valid report request should be acknowledged within a few hundred milliseconds, regardless of how long the report takes to generate.

Report generation should not depend on the HTTP connection remaining open for the duration of the work.

The client should be able to determine whether the report is complete, still in progress, or has failed without keeping the original connection alive.

The system should preserve the integrity and content of the generated report regardless of when the client disconnects or how long the work takes.

Report generation time should not determine how long the client must wait for a meaningful response.

The contract decision

The current contract promises completion in the original response. Based on your evidence and derived requirements, choose one option and justify it.

☐ Keep the current contract: return 200 OK with the completed report.

☐ Change the contract: acknowledge accepted work before the report is complete.

Justification:

The test results show that the response time increases as the report takes longer to generate. This means the client currently has to wait for the entire report to finish before receiving a response. While all tests were successful, a longer report could cause the connection to time out. By acknowledging the request first, the client would know it was received right away, while the report can continue generating in the background.

In-class exit ticket

Submit one response per pair before leaving.

Element	Your response
Claim	The synchronous API is not suitable for potentially long-running reports because response time is directly tied to report generation time; there is no way for the request to be acknowledged separately from completion.
Evidence	The mean response time ranged from approximately 0.0014s at 0s delay to 12.01s at 12s delay, tracking the artificial delay almost exactly across all four tested conditions.
Reasoning	The evidence follows the current contract because it requires the completed report to be returned in the same response that acknowledges the request; the handler cannot reply to the client until the report is fully generated. This causes the response time to equal the report-generation time, so the client has no way to know their request was received until the process is complete.
Requirement	The current contract cannot reliably satisfy the requirement; a valid report request should be acknowledged within a predictable time.

Complete outside of class

Submit a short laboratory report containing the following sections:

- Research question and hypothesis
- Independent, dependent, independent, and controlled variables
- Raw measurements and calculated mean response times
- Observations stated without explanation
- Causal explanation of the observed behavior
- At least two limitations of the experiment
- Requirements derived from the evidence
- A proposed revised contract

Required reasoning chain

Layer	Question to answer
Evidence	What did we measure? We measured that response time tracks artificial delay nearly 1:1 (0.0014s, 3.01s, 7.01s, 12.01s). Every run returned 200 OK with a complete report. No run failed, but at 12s, the client waited the full 12s for a response.
Problem	What coupling did the evidence expose? The synchronous contract couples the client's waiting time directly to the server's work duration. The client connection is kept open throughout report generation, so a slow report indicates a slow (or timed-out) client.
Requirement	What property does the system now need? The system needs to acknowledge receipt of a valid request quickly, regardless of how long the report takes to generate. The client should not have to maintain a live connection to receive a final result.
Contract	What should the API promise the client?

Layer	Question to answer
	The API should acknowledge accepted work immediately with an accepted status and provide a way for the client to retrieve the completed report later, rather than promising it in the original response.
Implementation	What mechanisms might provide that contract? A job resource with a unique identifier, an asynchronous worker that processes report generation, an immediate 202 Accepted acknowledgment, and a polling endpoint to check job status and retrieve results when ready.

Important distinctions

Incorrect conclusion	Stronger conclusion
Async makes report generation faster.	Async can separate acknowledgment time from completion time; the work may take just as long.
The design fails after exactly 10 seconds.	The observed threshold depends on configuration; the underlying issue is lifetime coupling.
The timeout is the whole problem.	A larger timeout preserves the synchronous contract and merely permits longer waiting.
202 means the report succeeded.	202 means the server accepted the work; completion or failure must be observed later.

Next laboratory, you will redesign the contract using Job + Worker + 202 Accepted + GET /jobs/{id}, then repeat the measurements and coacknowledgment time with completion time.

Reference:

Output used for the report - Christian:

0s

```
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=0.003167s
bytes=369
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=0.001771s
bytes=368
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=0.001441s
bytes=369
```

3s

```
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=3.008541s
bytes=369
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=3.005394s
bytes=369
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=3.002763s
bytes=369
```

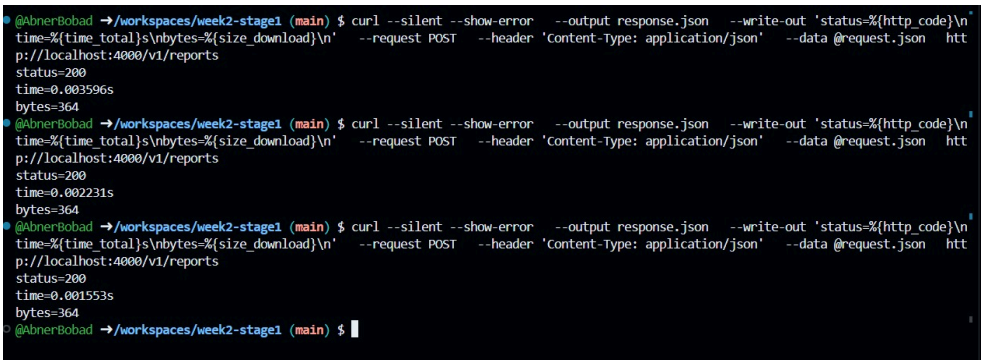
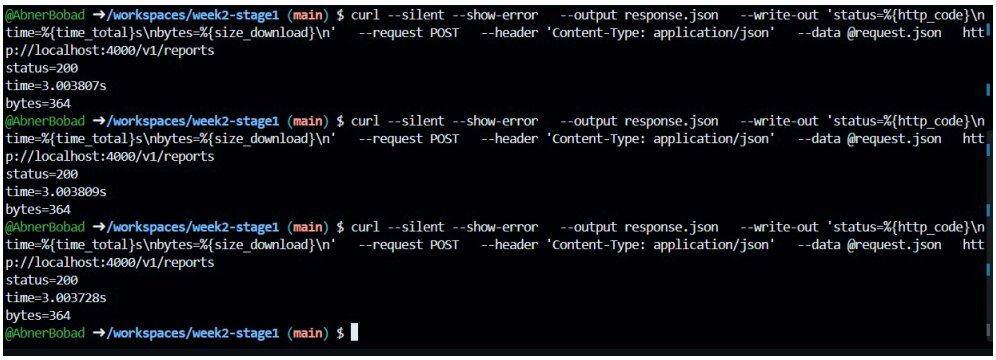
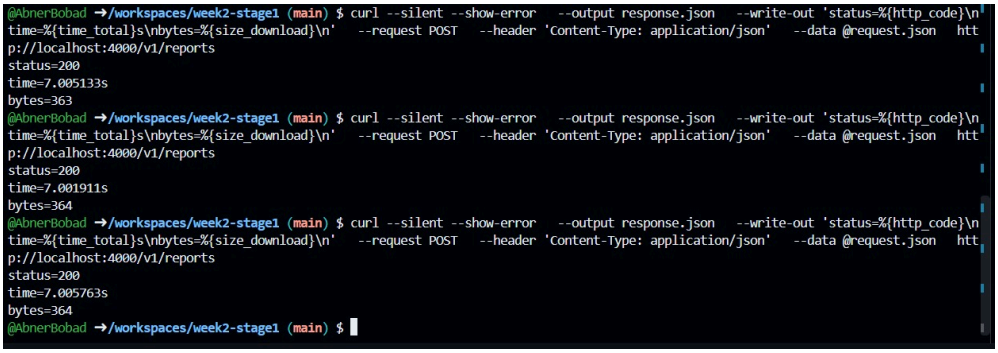
7s

```
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=7.012026s
bytes=369
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=7.009421s
bytes=368
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=7.009061s
bytes=368
```

12s

```
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=12.010484s
bytes=369
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=12.004982s
bytes=369
christian@Christian-Laptop:~/Adv Web/stage-1-synchronous$ curl --silent --show-error   --output response.json
 --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n'   --request POST   --header 'Co
ntent-Type: application/json'   --data @request.json   http://localhost:4000/v1/reports
status=200
time=12.003802s
bytes=368
```

Output - Abner:

0s	
3s	
7s	
12s